COMP 3920 Database Systems 1

Java Lab

Introduction:
The goal of this document is to serve as a walkthrough of connecting Java to MySQL 8 and MS SQL Server.

For this document we will be using IntelliJ as a Java GUI.

This application will have you create a simple Java console application that can connect to MySQL and MS SQL Server, run a simple SELECT statement and print out the results to the console.

Step 1:

Import the MySQL Database provided on the learning hub.

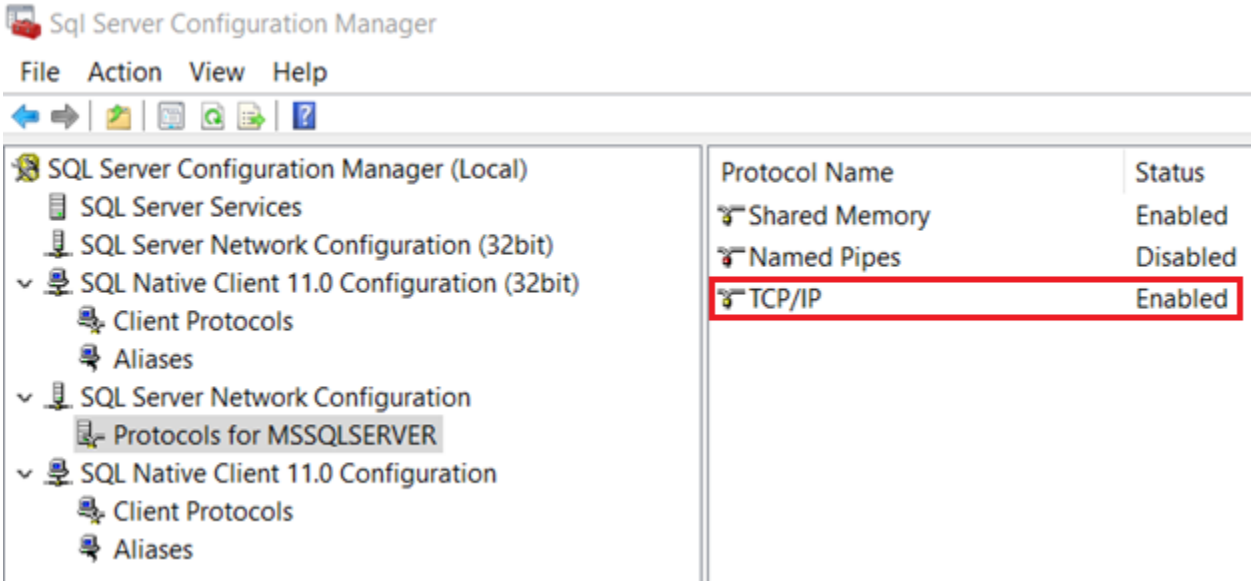
Step 2:

Import the MS SQL Database provided on the learning hub.

Step 3:

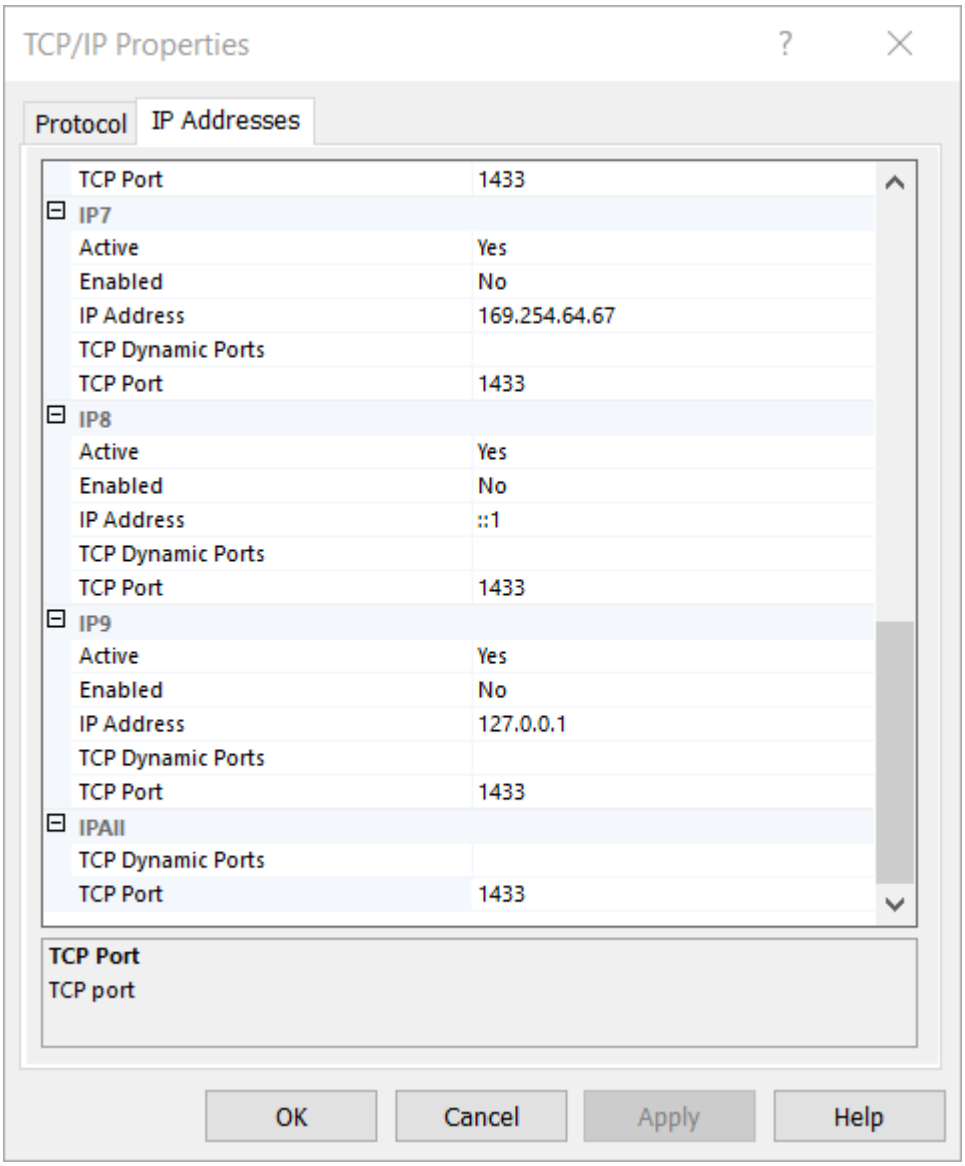
The default port for MS SQL Server is supposed to be 1433.
However, it might not be enabled by default.

To enable or change the external port of MS SQL Server, open the SQL Server Configuration Manager and make sure the Protocols for MSSQLSERVER -> TCP/IP are Enabled. (Right click on it and click Enable, if it is disabled).



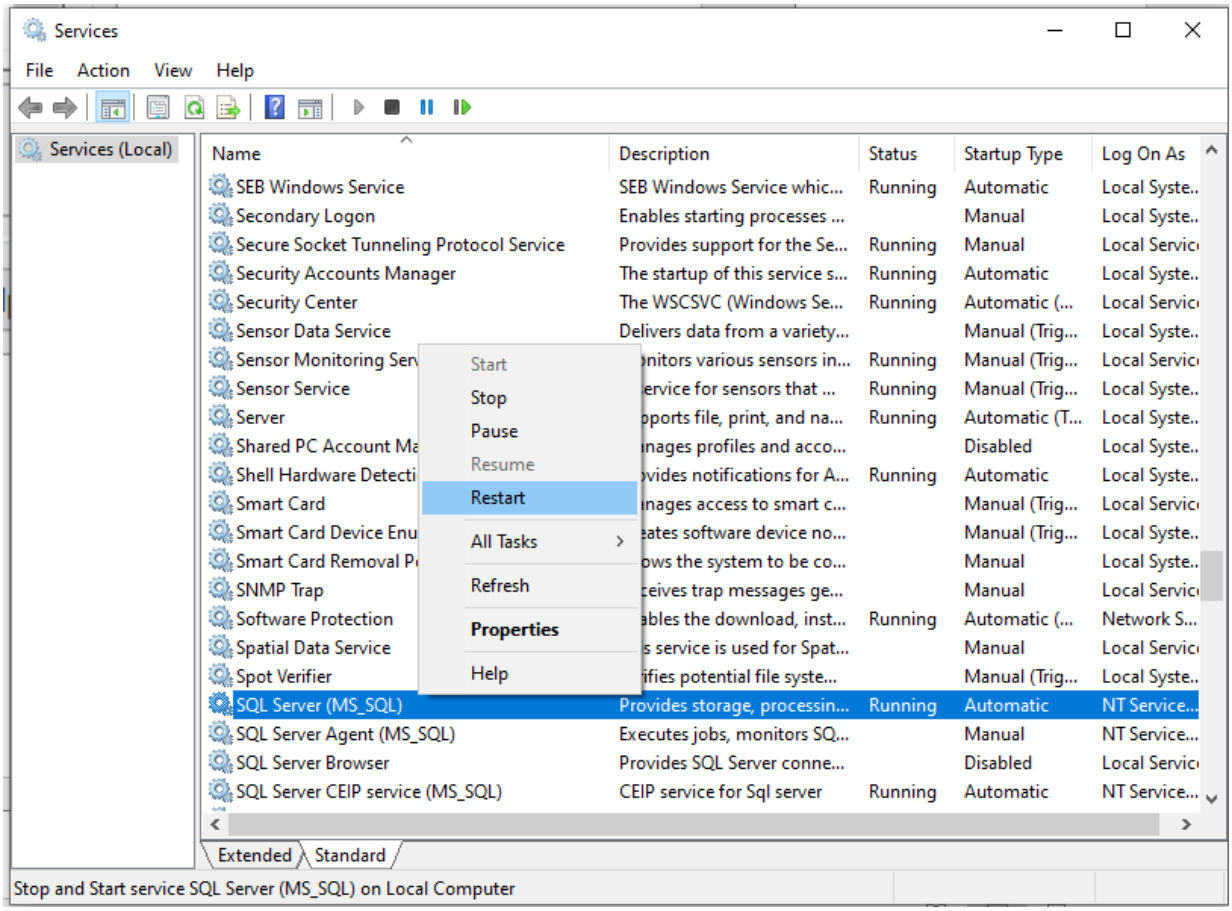
Right click on TCP/IP and click on the IP Addresses tab.

Find IPALL at the bottom of the list and ensure that the TCP Port is 1433.



Click OK and close the SQL Server Configuration Manager.

IMPORTANT: For any IP Address changes to take effect for the MS SQL Server Service, you need to restart the SQL Server (MS_SQL) Service within the Windows Services Manager (Search for Services in the Windows Start Menu)



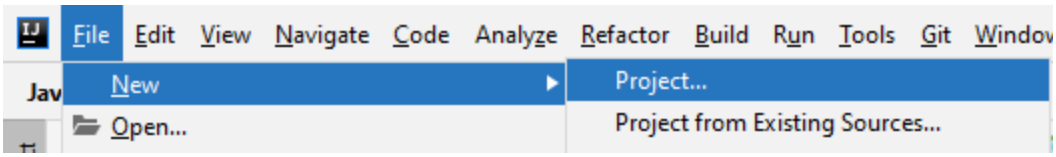
Step 4:
Create a new project in IntelliJ.

Open IntelliJ.

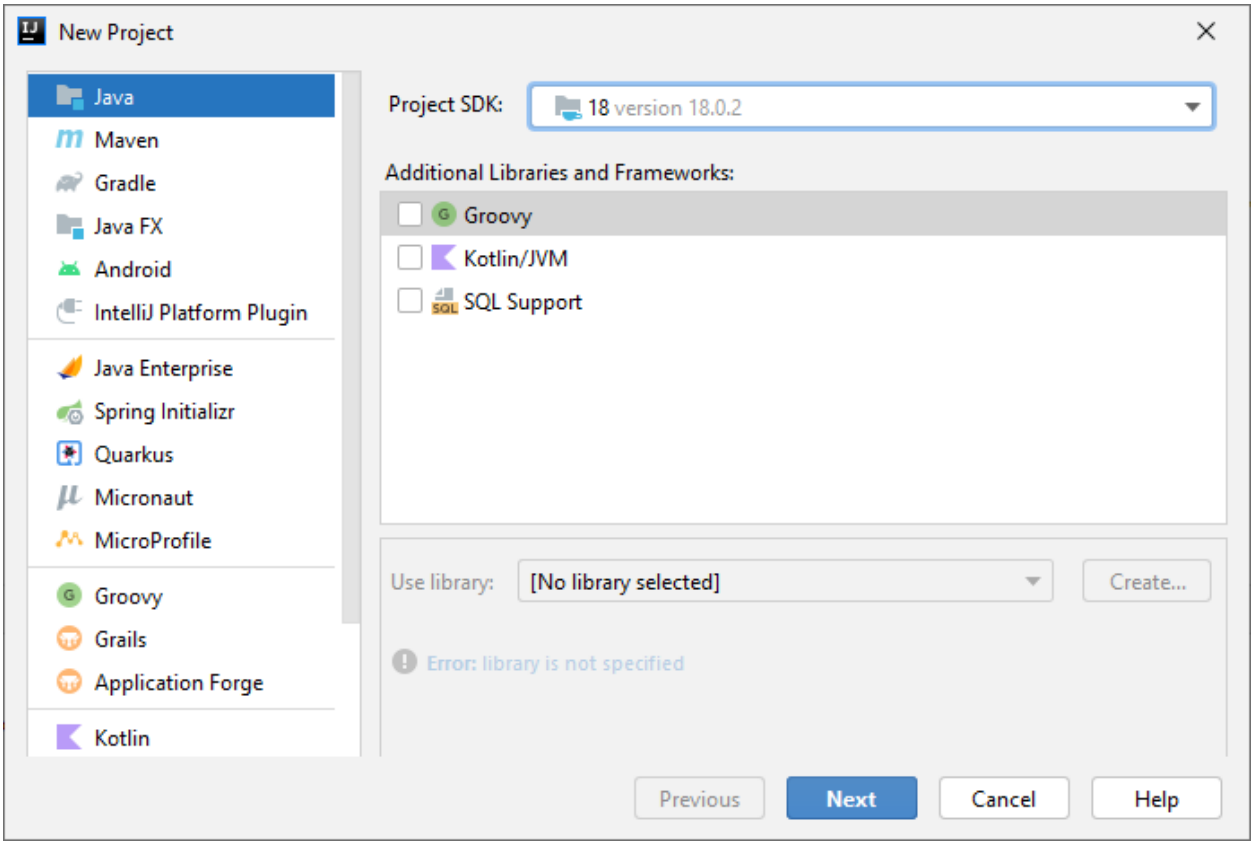
(If you don't have IntelliJ installed, it can be downloaded here: https://www.jetbrains.com/idea/download/)

Create a new Java project.

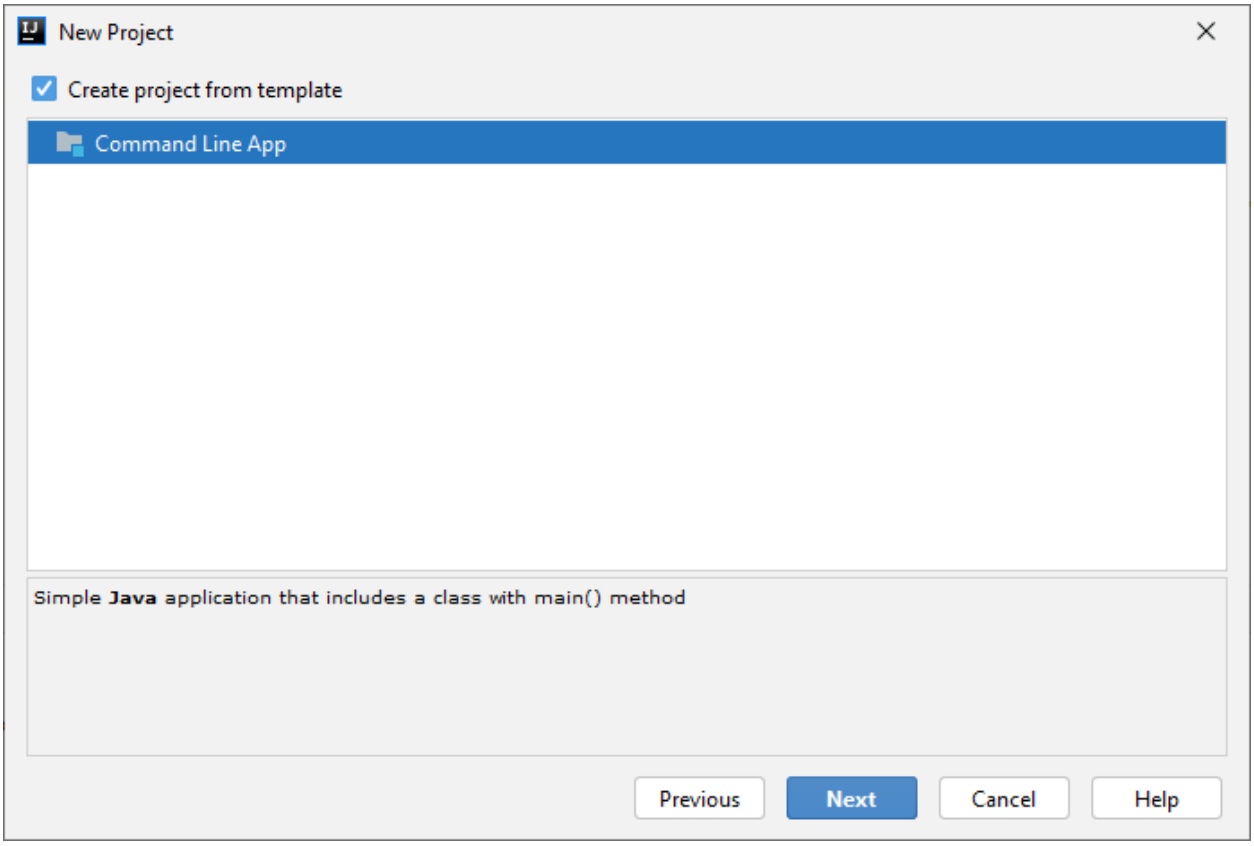
Click on File -> New -> Project…



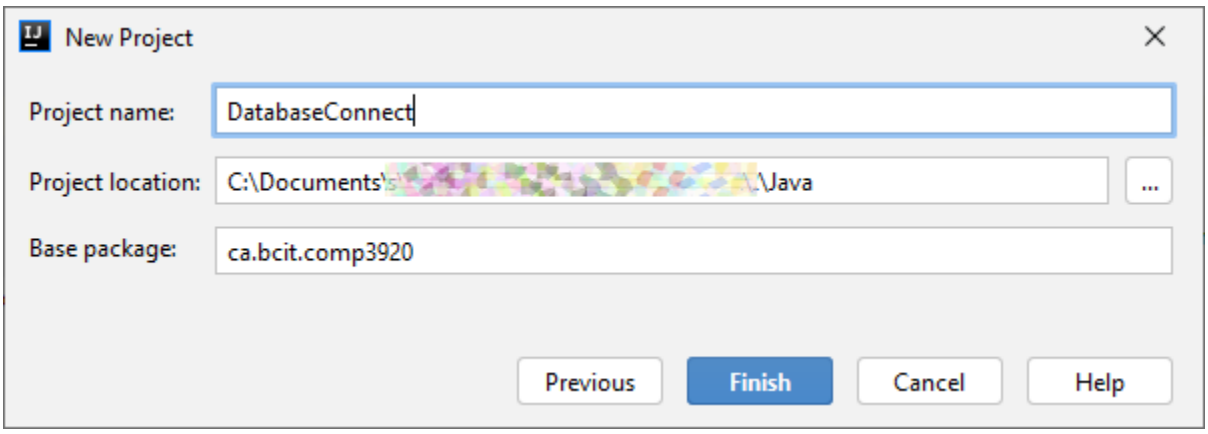
Select Java as the project type, and your Project JDK (I am selecting version 18).



Click Next



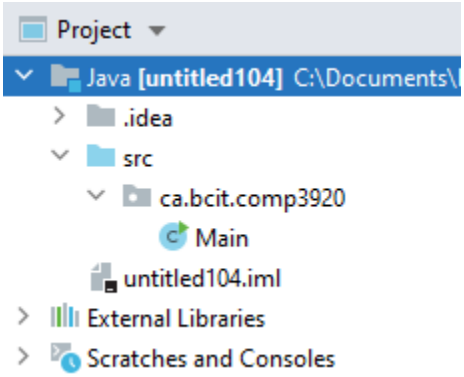
Click on Create from template, and select the "Command Line App" as a template.
Click Next.



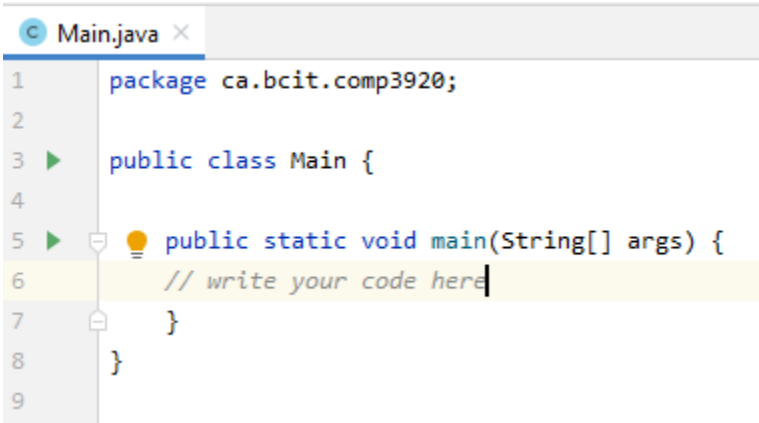
For the Project name you can use: "DatabaseConnect"
Pick a suitable folder on your local drive for the project location.
Use "ca.bcit.comp3920" as a base package.

Click Finish.

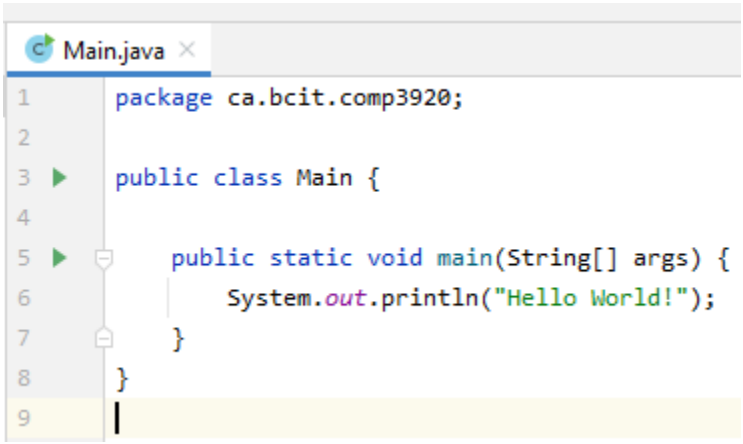
Your project explorer should look like this:

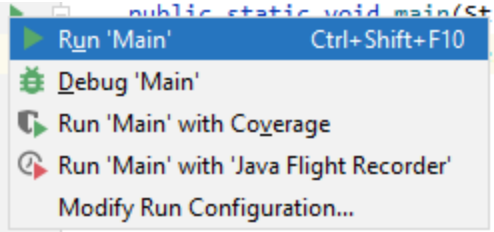


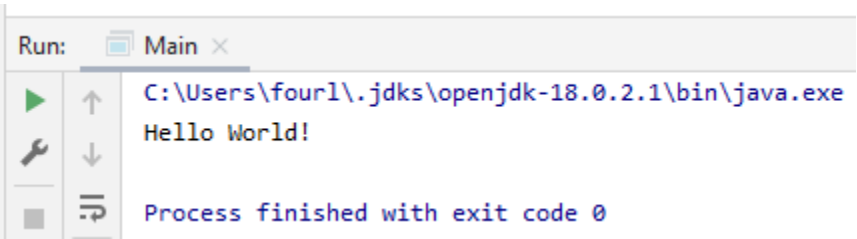
And your main class should look like this:



In your main method, add a `System.out.println()` which will output the typical `"Hello World!"` and run your application just to confirm that everything is working.

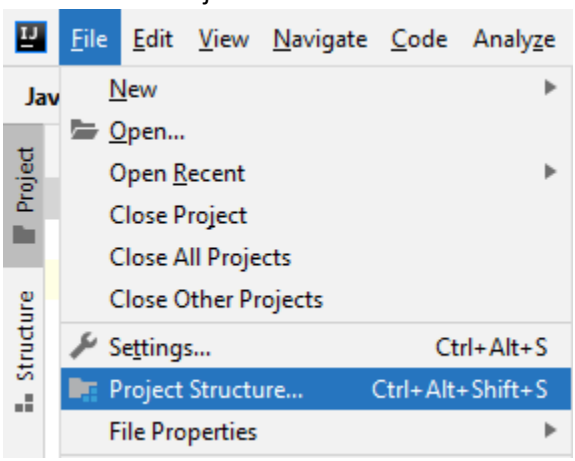




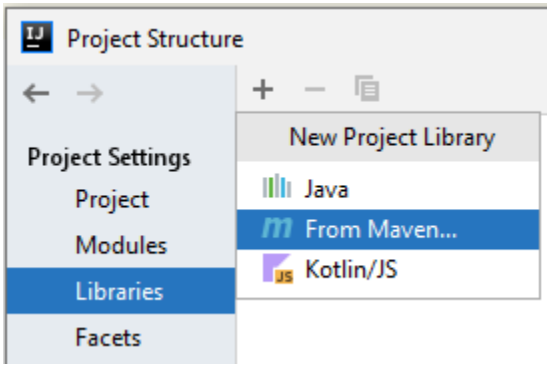


Step 5:
Install the library for MySQL Connector.

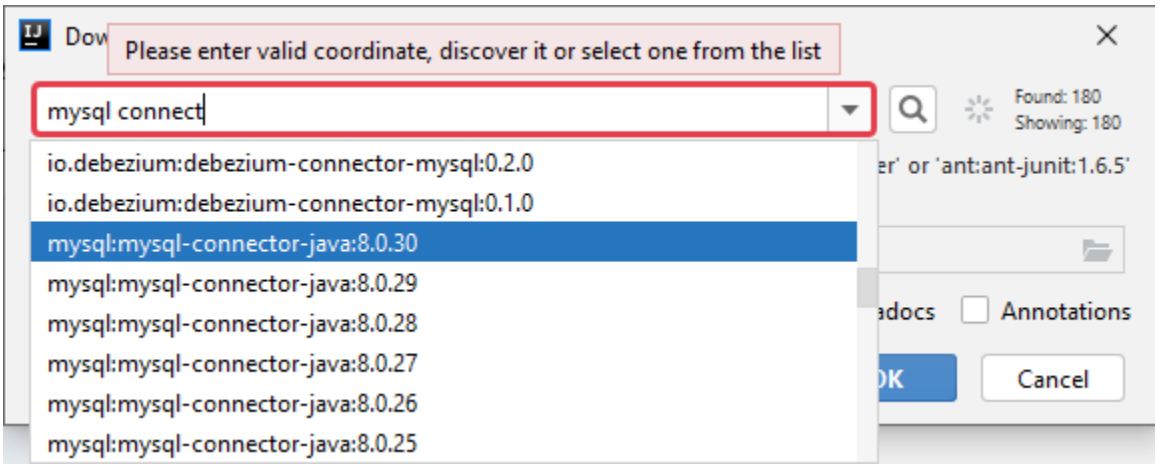
Under File -> Project Structure…



Find Library and Click on the plus symbol + to add "From Maven"



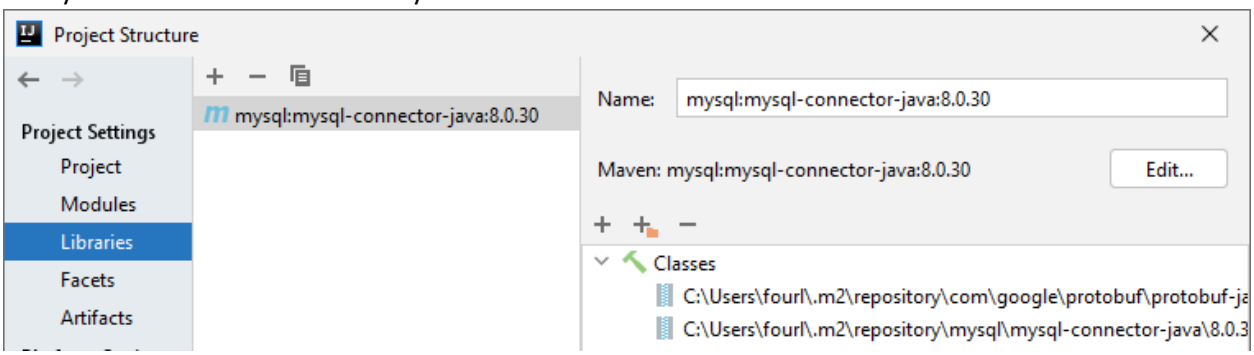
In the dropdown menu, type "mysql connect" and this should be enough to find: mysql:mysql-connector-java



Pick the latest version, mine is: mysql:mysql-connector-java:8.0.30.

Click OK.

Verify it's been added to the library list and Click OK.



Step 6:
Add the code to connect to MySQL

In order to connect to your MySQL Database, you'll need to provide the mysql connector a few pieces of information, such as:

- host
- port
- database
- username
- password

Host is the IP address (or a DNS that resolves to an IP address) of the database server.
This can be `localhost` if attempting to connect to your current machine. Or something like `mydomain.com` if this resolves to an IP address that has a MySQL instance running and publicly accessible. You can also specify the IP address directly as well, such as: 127.0.0.1 (for localhost) or 218.70.3.99 (an example of a public IP).

For the **Port**, MySQL typically runs on port `3306`. Unless your server is running on a non-standard port, which is typically very rare, you'll use port `3306`.

The **database** is the name of the database that you will be running your queries against. It is equivalent of connecting to MySQL and running `USE <database>;` (where `<database>` is the name of the database specified) – this sets the default database and all queries (unless otherwise specified) will be applied to this database and to the tables within it.

We also need to provide the authentication for the connection. The authentication includes the **user** and **password**. We will add the user and password to the front of the URL with a colon "`:`" separating the user and password and with an asperand (also called the *at* symbol) "`@`" before the host.

These 5 connection properties are often together as a connection string like this:
`jdbc:mysql://user:password1@localhost:3306/application_data`

In this example, we are connecting to the `localhost` on the standard port (`3306`) to a database called `application_data`. The username is `user` and the password is `password1`.

Add the following code within your `public static void main() method`:

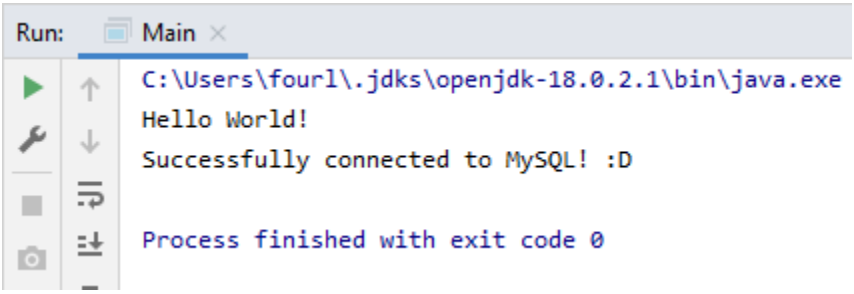
```
try {
    Class.forName("com.mysql.cj.jdbc.Driver");
} catch (ClassNotFoundException e) {
    System.out.println("Your MySQL JDBC Driver is not found. Include it in your
library path ");
    e.printStackTrace();
}

final String MYSQL_HOST = "localhost";
final int MYSQL_PORT = 3306;
final String MYSQL_DATABASE_NAME = "comp3920";
final String MYSQL_USER_NAME = "root";
final String MYSQL_PASSWORD = "YourPasswordHere";
final String MYSQL_URL = "jdbc:mysql://"+ MYSQL_USER_NAME+":"+
MYSQL_PASSWORD+"@"+ MYSQL_HOST+":"+ MYSQL_PORT+"/" + MYSQL_DATABASE_NAME;

final Connection connection;
try {
    connection = DriverManager.getConnection(MYSQL_URL);
}
catch (SQLException e) {
    System.out.println("Connection Failed! Check output console");
    e.printStackTrace();
    return;
}

if (connection != null) {
    System.out.println("Successfully connected to MySQL! :D");
}
```

Replace **"YourPasswordHere"** with the password for your `root` account in MySQL.

We'll also need the java.sql imports:

```
import java.sql.*;
```

Run your code again and you should get `Successfully connected to MySQL! :D` as output.



Step 7:
Print out some data.

Ok, so we have shown we can connect to the database, but we haven't really done anything yet. Let's write a simple SELECT statement to show the contents of a table.

Add the following code after your connection code in the main method:

```
final String SELECT_STATEMENT = """
    SELECT address.*, address_type.type
    FROM address
    JOIN address_type ON address.address_type_id = address_type.address_type_id
    """;
try {

    PreparedStatement stmt = connection.prepareStatement(SELECT_STATEMENT,
ResultSet.TYPE_SCROLL_INSENSITIVE, ResultSet.CONCUR_READ_ONLY);

    ResultSet resultSet = stmt.executeQuery();

    printResultSet(resultSet);
}
catch (SQLException e) {
    System.out.println("Error executing query");
    e.printStackTrace();
}
```

And add this static method after the main method:

```
public static void printResultSet(ResultSet resultSet) throws SQLException {
    ResultSetMetaData rsmd = resultSet.getMetaData();
    int columnsNumber = rsmd.getColumnCount();
    int rows = 0;
    while (resultSet.next()) {
        rows++;
        if (rows==1) {
            for (int col = 1; col <= columnsNumber; col++) {
                if (col > 1) {
                    System.out.print(",\t\t");
                }
                System.out.print(rsmd.getColumnName(col));
            }
            System.out.println("");
        }
        for (int col = 1; col <= columnsNumber; col++) {
            if (col > 1) {
                System.out.print(",\t\t");
            }
            String columnValue = resultSet.getString(col);
            System.out.print(columnValue);
        }
        System.out.println("");
    }
}
```

This code does a number of things, so let's talk about it.

First we will declare a String variable that contains the SELECT statement we want to run. I've used the triple double quotation marks (`"""`) here to make it easier to format my SELECT statement on multiple lines without having to use escape characters.

Next, we "prepare" the statement. This just tells the MySQL drive we will be using this SELECT statement a query. It can also help us prevent SQL Injection attacks, but more on this later.

As part of this `prepareStatement` call we specify extra options like `ResultSet.`*`TYPE_SCROLL_INSENSITIVE`* and `ResultSet.`*`CONCUR_READ_ONLY`*.

The first option is the scroll type. I've specified *`TYPE_SCROLL_INSENSITIVE`* as a scroll type. This lets us iterate over the resultset multiple times. This opposed to the default which is *`TYPE_SCROLL_FORWARD`* (which only allows to iterate of the rows in the resultset once).

The second option is concurrency, and involves whether or not the resultset returned is editable or not. I've explicitly using the default which is *`CONCUR_READ_ONLY`*. While there are a few rare cases where you might want to use *`CONCUR_UPDATABLE`* I don't see it being used very often. Using the updatable option would let you call `resultSet.updateRow()` and it would create an UPDATE statement which gets sent back to the server and executed. The problem with this, is that many (if not most) query resultsets are non-updatable. Any query that SELECTs from multiple tables in a JOIN or UNION; any

aggregate query with a GROUP BY, or a query that doesn't include a unique key for the table will NOT be updatable.

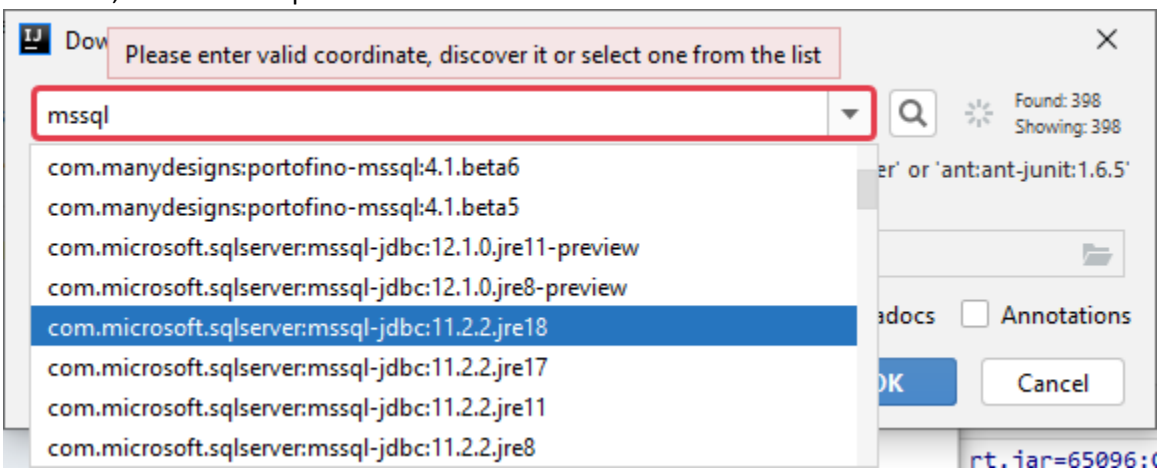
Next, we execute the query using `stmt.executeQuery()` and call the method `printResultSet()` to display the results.
Displaying the results uses a few lines of code, but is relatively simple to understand. We first capture the resultset metadata which contains the names of the columns so we can print them out as column headers. Next, we output each row, column by column. Each column's output is separated by a comma and tabs `"\t\t"` and each row is separated by a new line.

Step 8:
Connect to Microsoft SQL Server

Connecting and printing data from MS SQL Server is similar to MySQL. The main difference is the connection URL, the authentication, and library. We first need the MSSQL JDBC Driver.

Just like we did for the MySQL Connection driver, we can bring in the MSQL Driver.

This time, look for "mssql" and find the official one from Microsoft.



In my case, the latest version for my Java Runtime is:
`com.microsoft.sqlserver:mssql-jdbc:11.2.2.jre18`.

And now you are ready to paste the following code into your main method:

```
try {
    Class.forName("com.microsoft.sqlserver.jdbc.SQLServerDriver");
}
catch(ClassNotFoundException e) {
    System.out.println("Your MS SQL JDBC Driver is not found. Include it in
your library path.");
    e.printStackTrace();
}

final String MSSQL_HOST = "localhost";
final int MSSQL_PORT = 1433;
final String MSSQL_DATABASE_NAME = "comp3920";

final String MSSQL_USER_NAME = "sa";
final String MSSQL_PASSWORD = "YourPasswordHere";

final String MSSQL_URL = "jdbc:sqlserver://"
        + MSSQL_HOST + ":" + MSSQL_PORT + ";"
        + "database=" + MSSQL_DATABASE_NAME + ";"
        + "user=" + MSSQL_USER_NAME + ";"
        + "password=" + MSSQL_PASSWORD + ";"
        + "encrypt=true;"
        + "trustServerCertificate=true;"
        + "loginTimeout=30;";
final Connection connection2;
try {
    connection2 = DriverManager.getConnection(MSSQL_URL);
}
catch (SQLException e) {
    System.out.println("Connection Failed! Check output console");
    e.printStackTrace();
    return;
}

if (connection2 != null) {
    System.out.println("Successfully connected to MS SQL! :D ");
}
```

Replace **"YourPasswordHere"** with the password for your `sa` account in MS SQL.

If, to you, this code looks similar to the code we used for MySQL, you'd be right! There are a few differences, though that are important to point out. The default port for a secured connection to MS SQL is `1433` (remember this was `3306` for MySQL). Notice too that the default user for MS SQL is `sa` whereas MySQL had `root`. Notice also the format for the connection `URL` is slightly different. While it contains the same 5 elements (host, port, database name, username, and password), Microsoft has chosen to use a different format using sets of parameters with equals signs, separated by semicolons. The rest of the code, however, should be essentially the same.

Once we have the libraries loaded and our connections established, the code for querying the database is mostly the same.

```
try {

    PreparedStatement stmt = connection2.prepareStatement(SELECT_STATEMENT,
ResultSet.TYPE_SCROLL_INSENSITIVE, ResultSet.CONCUR_READ_ONLY);

    ResultSet resultSet = stmt.executeQuery();

    printResultSet(resultSet);
}
catch (SQLException e) {
    System.out.println("Error executing query");
    e.printStackTrace();
}
```

The only difference here is that we are using `connection2` (not `connection`) which is setup to use our MS SQL Server.

Submit your code:
Filename: **Main.java**

Submit

Main.java

```
package ca.bcit.comp3920;

import java.sql.*;

public class Main {

    public static void main(String[] args) {
        System.out.println("Hello World!");

        try {
            Class.forName("com.mysql.cj.jdbc.Driver");
        } catch (ClassNotFoundException e) {
            System.out.println("Your MySQL JDBC Driver is not found. Include it in your library path ");
            e.printStackTrace();
        }
```